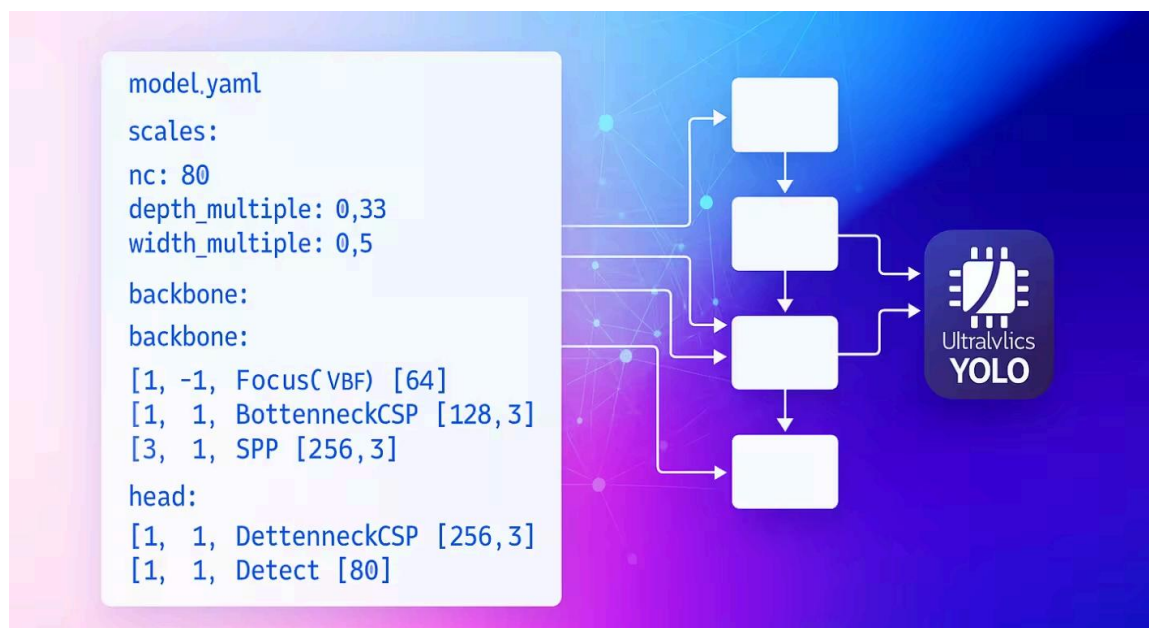


模型 YAML 配置文件指南

模型 YAML 配置文件是 Ultralytics 神经网络的架构蓝图。它定义了层如何连接、每个模块使用什么参数，以及整个网络如何在不同模型尺寸上缩放。



配置结构

模型 YAML 文件分为三个主要部分，它们协同工作以定义架构。

参数部分

parameters 部分指定模型的全局特性和缩放行为：

```
# Parameters
nc: 80 # number of classes
scales: # compound scaling constants [depth, width, max_channels]
  n: [0.50, 0.25, 1024] # nano: shallow layers, narrow channels
  s: [0.50, 0.50, 1024] # small: shallow depth, standard width
  m: [0.50, 1.00, 512] # medium: moderate depth, full width
  l: [1.00, 1.00, 512] # large: full depth and width
  x: [1.00, 1.50, 512] # extra-large: maximum performance
kpt_shape: [17, 3] # pose models only
```

Ask AI 

- **nc** 设置模型预测的类别数。

- `scales` 定义复合缩放因子，用于调整模型深度、宽度和最大通道数，以生成不同尺寸的变体（从 nano 到 extra-large）。
- `kpt_shape` 适用于姿势估计模型。它可以是 `[N, 2]` 用于 `(x, y)` 关键点或 `[N, 3]` 用于 `(x, y, visibility)`。

 使用以下方法减少冗余 `scales`

字段 `scales` parameter lets you generate multiple model sizes from a single base YAML. For instance, when you load `yolo11n.yaml` 时，Ultralytics 读取基本 `yolo11.yaml` 并应用 `n` 缩放因子 (`depth=0.50`, `width=0.25`) 来构建 nano 变体。

 `nc` 和 `kpt_shape` 是数据集相关的

如果您的数据集指定了不同的 `nc` 或 `kpt_shape`，Ultralytics 将在运行时自动覆盖模型配置以匹配数据集 YAML。

Backbone 和 Head 架构

模型架构由主干（特征提取）和头部（特定任务）部分组成：

```
backbone:
  # [from, repeats, module, args]
  - [-1, 1, Conv, [64, 3, 2]] # 0: Initial convolution
  - [-1, 1, Conv, [128, 3, 2]] # 1: Downsample
  - [-1, 3, C2f, [128, True]] # 2: Feature processing

head:
  - [-1, 1, nn.Upsample, [None, 2, nearest]] # 6: Upsample
  - [[-1, 2], 1, Concat, [1]] # 7: Skip connection
  - [-1, 3, C2f, [256]] # 8: Process features
  - [[8], 1, Detect, [nc]] # 9: Detection layer
```

层规范格式

每个层都遵循一致的模式： `[from, repeats, module, args]`

组件	目的	实例
从	输入连接	<code>-1</code> （前一层）， <code>6</code> （第 6 层）， <code>[4, 6, 8]</code> （多输入）
重复次数	重复次数	<code>1</code> （单次）， <code>3</code> （重复 3 次）

组件	目的	实例
模块	模块类型	Conv, C2f, TorchVision, Detect
args	模块参数	[64, 3, 2] （通道数、卷积核大小、步长）

连接模式

字段 `from` 字段可在整个网络中创建灵活的数据流模式：

顺序流

```
- [-1, 1, Conv, [64, 3, 2]] # Takes input from previous layer
```

跳跃连接

```
- [[-1, 6], 1, Concat, [1]] # Combines current layer with layer 6
```

多输入融合

```
- [[4, 6, 8], 1, Detect, [nc]] # Detection head using 3 feature scales
```

 层索引

层索引从 0 开始。负索引引用先前的层（-1 = 前一层），而正索引按位置引用特定层。

模块重复

字段 `repeats` 参数创建更深的网络部分：

```
- [-1, 3, C2f, [128, True]] # Creates 3 consecutive C2f blocks
- [-1, 1, Conv, [64, 3, 2]] # Single convolution layer
```

实际重复计数乘以模型大小配置中的深度缩放因子。

可用模块

模块按功能组织，并在 [Ultralytics 模块目录](#) 中定义。下表显示了按类别划分的常用模块，源代码中提供了更多模块：

基本操作

模块	目的	来源	参数
Conv	卷积 + BatchNorm + 激活函数	conv.py	[out_ch, kernel, stride, pad, groups]
nn.Upsample	空间上采样	PyTorch	[size, scale_factor, mode]
nn.Identity	Pass-through 操作	PyTorch	[]

复合块

模块	目的	来源	参数
C2f	带有 2 个卷积的 CSP 瓶颈	block.py	[out_ch, shortcut, expansion]
SPPF	空间金字塔池化（快速）	block.py	[out_ch, kernel_size]
Concat	通道式连接	conv.py	[dimension]

专用模块

模块	目的	来源	参数
TorchVision	加载任何 torchvision 模型	block.py	[out_ch, model_name, weights, unwrap, truncate, split]
Index	从列表中提取特定 tensor	block.py	[out_ch, index]
Detect	YOLO 检测头	head.py	[nc, anchors, ch]

 完整模块列表

这代表了可用模块的一个子集。有关模块及其参数的完整列表，请浏览[modules 目录](#)。

高级功能

TorchVision 集成

TorchVision 模块支持将任何 [TorchVision 模型](#) 无缝集成为骨干网络：

Python

```
from ultralytics import YOLO

# Model with ConvNeXt backbone
model = YOLO("convnext_backbone.yaml")
results = model.train(data="coco8.yaml", epochs=100)
```

YAML 配置

```
backbone:
  - [-1, 1, TorchVision, [768, convnext_tiny, DEFAULT, True, 2, False]]
head:
  - [-1, 1, Classify, [nc]]
```

参数细分：

- 768：预期输出通道数
- convnext_tiny：模型架构 ([可用模型](#))
- DEFAULT：使用预训练权重
- True：移除分类头
- 2：截断最后 2 层
- False：返回单个 tensor (非列表)

多尺度特征

将最后一个参数设置为 True 以获取用于多尺度检测的中间特征图。

用于特征选择的 Index 模块

当使用输出多个特征图的模型时，Index 模块会选择特定的输出：

```
backbone:
  - [-1, 1, TorchVision, [768, convnext_tiny, DEFAULT, True, 2, True]] #
Multi-output
head:
  - [0, 1, Index, [192, 4]] # Select 4th feature map (192 channels)
  - [0, 1, Index, [384, 6]] # Select 6th feature map (384 channels)
  - [0, 1, Index, [768, 8]] # Select 8th feature map (768 channels)
  - [[1, 2, 3], 1, Detect, [nc]] # Multi-scale detection
```

模块解析系统

了解 Ultralytics 如何定位和导入模块对于自定义至关重要：

模块查找过程

Ultralytics 在以下位置使用三层系统 `parse_model`：

```
# Core resolution logic
m = getattr(torch.nn, m[3:]) if "nn." in m else getattr(torchvision.ops,
m[4:]) if "ops." in m else globals()[m]
```

1. **PyTorch 模块**：名称以 `'nn.'` → `torch.nn` 命名空间
2. **TorchVision 操作**：名称以 `'ops.'` → `torchvision.ops` 命名空间
3. **Ultralytics 模块**：所有其他名称 → 通过导入的全局命名空间

模块导入链

标准模块通过在以下位置导入变得可用 `tasks.py`：

```
from ultralytics.nn.modules import ( # noqa: F401, E501
    SPPF,
    C2f,
    Conv,
    Detect,
    # ... many more modules
    Index,
    TorchVision,
)
```

自定义模块集成

源代码修改

修改源代码是集成自定义模块的最通用方法，但它可能很棘手。要定义和使用自定义模块，请按照以下步骤操作：

1. 使用 [快速入门指南](#) 中的 Git 克隆方法在开发模式下安装 Ultralytics。
2. 定义您的模块 在 `ultralytics/nn/modules/block.py`：

```
class CustomBlock(nn.Module):
    """Custom block with Conv-BatchNorm-ReLU sequence."""

    def __init__(self, c1, c2):
```

```

        """Initialize CustomBlock with input and output channels."""
        super().__init__()
        self.layers = nn.Sequential(nn.Conv2d(c1, c2, 3, 1, 1),
                                     nn.BatchNorm2d(c2), nn.ReLU())

    def forward(self, x):
        """Forward pass through the block."""
        return self.layers(x)

```

3. 在包级别公开您的模块 在 `ultralytics/nn/modules/__init__.py`:

```

from .block import CustomBlock # noqa makes CustomBlock available as
ultralytics.nn.modules.CustomBlock

```

4. 添加到导入 在 `ultralytics/nn/tasks.py`:

```

from ultralytics.nn.modules import CustomBlock # noqa

```

5. 处理特殊参数（如果需要）在内部 `parse_model()` 在 `ultralytics/nn/tasks.py`:

```

# Add this condition in the parse_model() function
if m is CustomBlock:
    c1, c2 = ch[f], args[0] # input channels, output channels
    args = [c1, c2, *args[1:]]

```

6. 在您的模型 YAML 中使用该模块:

```

# custom_model.yaml
nc: 1
backbone:
  - [-1, 1, CustomBlock, [64]]
head:
  - [-1, 1, Classify, [nc]]

```

7. 检查 FLOPs 以确保正向传递有效:

```

from ultralytics import YOLO

model = YOLO("custom_model.yaml", task="classify")
model.info() # should print non-zero FLOPs if working

```

配置示例

基本检测模型

```
# Simple YOLO detection model
nc: 80
scales:
  n: [0.33, 0.25, 1024]

backbone:
  - [-1, 1, Conv, [64, 3, 2]] # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]] # 1-P2/4
  - [-1, 3, C2f, [128, True]] # 2
  - [-1, 1, Conv, [256, 3, 2]] # 3-P3/8
  - [-1, 6, C2f, [256, True]] # 4
  - [-1, 1, SPPF, [256, 5]] # 5

head:
  - [-1, 1, Conv, [256, 3, 1]] # 6
  - [[6], 1, Detect, [nc]] # 7
```

TorchVision Backbone 模型

```
# ConvNeXt backbone with YOLO head
nc: 80

backbone:
  - [-1, 1, TorchVision, [768, convnext_tiny, DEFAULT, True, 2, True]]

head:
  - [0, 1, Index, [192, 4]] # P3 features
  - [0, 1, Index, [384, 6]] # P4 features
  - [0, 1, Index, [768, 8]] # P5 features
  - [[1, 2, 3], 1, Detect, [nc]] # Multi-scale detection
```

分类模型

```
# Simple classification model
nc: 1000

backbone:
  - [-1, 1, Conv, [64, 7, 2, 3]]
  - [-1, 1, nn.MaxPool2d, [3, 2, 1]]
  - [-1, 4, C2f, [64, True]]
  - [-1, 1, Conv, [128, 3, 2]]
  - [-1, 8, C2f, [128, True]]
  - [-1, 1, nn.AdaptiveAvgPool2d, [1]]

head:
  - [-1, 1, Classify, [nc]]
```

最佳实践

架构设计技巧

从简单开始: 在自定义之前, 从经过验证的架构开始。使用现有的 YOLO 配置作为模板, 并逐步修改, 而不是从头开始构建。

逐步测试: 逐步验证每个修改。一次添加一个自定义模块, 并在继续进行下一个更改之前验证它是否有效。

监控通道: 确保连接层之间的通道维度匹配。输出通道 (c2) 必须与输入通道 (c1) 即序列中下一层的通道数。

使用跳跃连接: 利用以下方式实现特征重用 `[[ -1, N], 1, Concat, [1]]` 模式。这些连接有助于梯度流动, 并允许模型组合来自不同尺度的特征。

适当缩放: 根据您的计算约束选择模型比例。对边缘设备使用 nano (n), 对平衡性能使用 small (s), 对最大精度使用更大的比例 (m, l, x)。

性能考量

深度 vs 宽度: 深层网络通过多个转换层捕获复杂的层次特征, 而宽层网络在每一层并行处理更多信息。根据您的任务复杂性来平衡这些。

跳跃连接: 改善训练期间的梯度流动, 并支持整个网络中的特征重用。它们在更深层的架构中尤为重要, 以防止梯度消失。

瓶颈块: 降低计算成本, 同时保持模型的表达能力。诸如 `c2f` 之类的模块使用比标准卷积更少的参数, 同时保留特征学习能力。

多尺度特征: 对于检测同一图像中不同大小的物体至关重要。将特征金字塔网络 (FPN) 模式与不同尺度的多个检测头一起使用。

故障排除

常见问题

问题	原因	解决方案
<code>KeyError: 'ModuleName'</code>	模块未导入	添加到 <code>tasks.py</code> 导入
通道维度不匹配	不正确的 <code>args</code> 规范	验证输入/输出通道兼容性
<code>AttributeError: 'int' object has no attribute</code>	错误的参数类型	检查模块文档以获取正确的参数类型

问题	原因	解决方案
模型构建失败	无效 from 参考	确保引用的层存在

调试技巧

在开发自定义架构时，系统性的调试有助于尽早发现问题：

使用 Identity Head 进行测试

用以下内容替换复杂的 head： `nn.Identity` 以隔离 backbone 问题：

```
nc: 1
backbone:
  - [-1, 1, CustomBlock, [64]]
head:
  - [-1, 1, nn.Identity, []] # Pass-through for debugging
```

这允许直接检查 backbone 输出：

```
import torch

from ultralytics import YOLO

model = YOLO("debug_model.yaml")
output = model.model(torch.randn(1, 3, 640, 640))
print(f"Output shape: {output.shape}") # Should match expected dimensions
```

模型架构检查

检查 FLOPs 计数并打印出每一层也有助于调试自定义模型配置的问题。有效模型的 FLOPs 计数应为非零。如果为零，则正向传递可能存在问题。运行简单的正向传递应显示遇到的确切错误。

```
from ultralytics import YOLO

# Build model with verbose output to see layer details
model = YOLO("debug_model.yaml", verbose=True)

# Check model FLOPs. Failed forward pass causes 0 FLOPs.
model.info()

# Inspect individual layers
for i, layer in enumerate(model.model.model):
    print(f"Layer {i}: {layer}")
```

逐步验证

1. **从最小开始**：首先使用最简单的架构进行测试
2. **逐步增加**：逐层构建复杂性
3. **检查尺寸**：验证通道和空间尺寸的兼容性
4. **验证缩放**：使用不同的模型比例进行测试（`n`，`s`，`m`）

常见问题

如何更改模型中的类别数量？

设置 `nc` YAML 文件顶部的参数以匹配数据集的类别数。

```
nc: 5 # 5 classes
```

我可以在我的模型 YAML 中使用自定义骨干网络吗？

是的，您可以使用任何受支持的模块，包括 TorchVision backbones，或者定义您自己的自定义模块并按照[自定义模块集成](#)中的描述导入它。

如何针对不同尺寸（nano、small、medium 等）缩放我的模型？

使用 `scales` 部分在 YAML 中定义深度、宽度和最大通道的缩放因子。当您加载基本 YAML 文件并将比例附加到文件名时（例如，`yolo11n.yaml`）。

是什么意思？`[from, repeats, module, args]` 格式是什么意思？

此格式指定了每一层的构建方式：

- `from`：输入源
- `repeats`：模块重复的次数
- `module`：层类型
- `args`：模块的参数

如何解决通道不匹配错误？

检查一层的输出通道是否与下一层的预期输入通道匹配。使用 `print(model.model.model)` 来检查您的模型架构。

在哪里可以找到可用模块及其参数的列表？

在以下位置检查源代码 `ultralytics/nn/modules` 目录下 以获取所有可用模块及其参数。

如何向我的 YAML 配置文件添加自定义模块？

在源代码中定义您的模块，如[源代码修改](#)中所示导入它，并在您的YAML文件中按名称引用它。

我可以在自定义 YAML 中使用预训练权重吗？

是的，您可以使用 `model.load("path/to/weights")` 从预训练的检查点加载权重。但是，只有匹配的层的权重才能成功加载。

如何验证我的模型配置？

使用 `model.info()` 检查FLOPs计数是否非零。有效的模型应显示非零FLOPs计数。如果为零，请按照中的建议进行操作 [调试技巧](#) 以查找问题。



创建于 1 个月前



更新于 12 天前



Tweet

分享